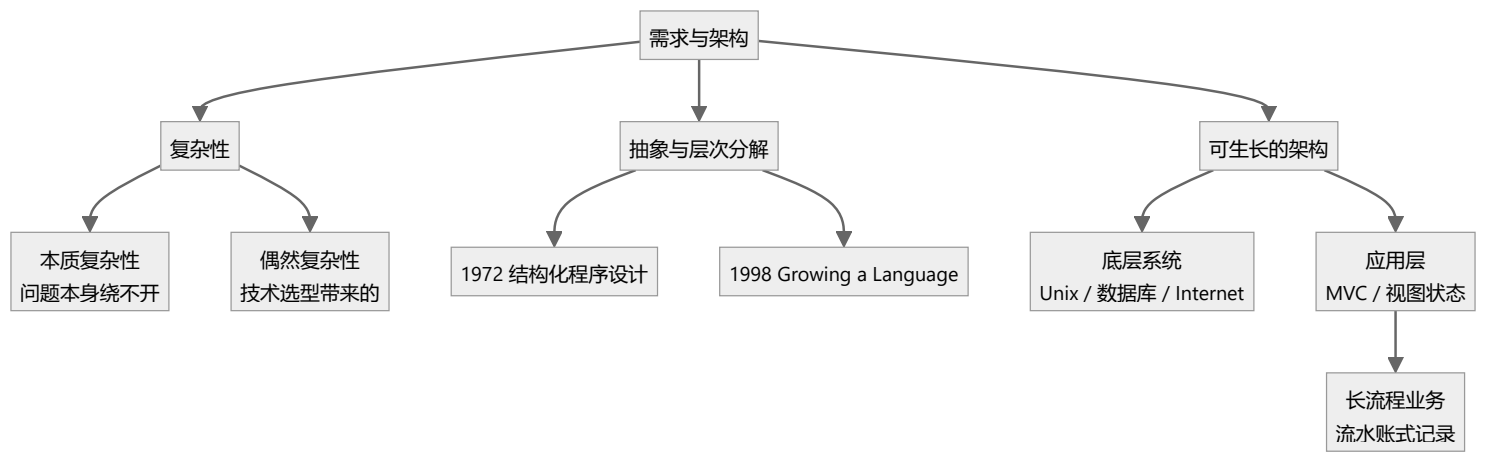


需求和架构（2）：复杂性、抽象与可生长的架构

这一讲要讲什么

上一讲用一份"需求规格说明书"说明了软件世界的复杂性。这一讲往前推一步：面对这些复杂性，架构到底能做什么。

整讲的内容可以分成三层：先复习复杂性本身，再回到 1970 到 1990 年代的老论文，看当时的人怎么想"人脑该怎么对付复杂系统"，最后落到几个经典的架构案例，以及它们在今天的应用层怎么演化。



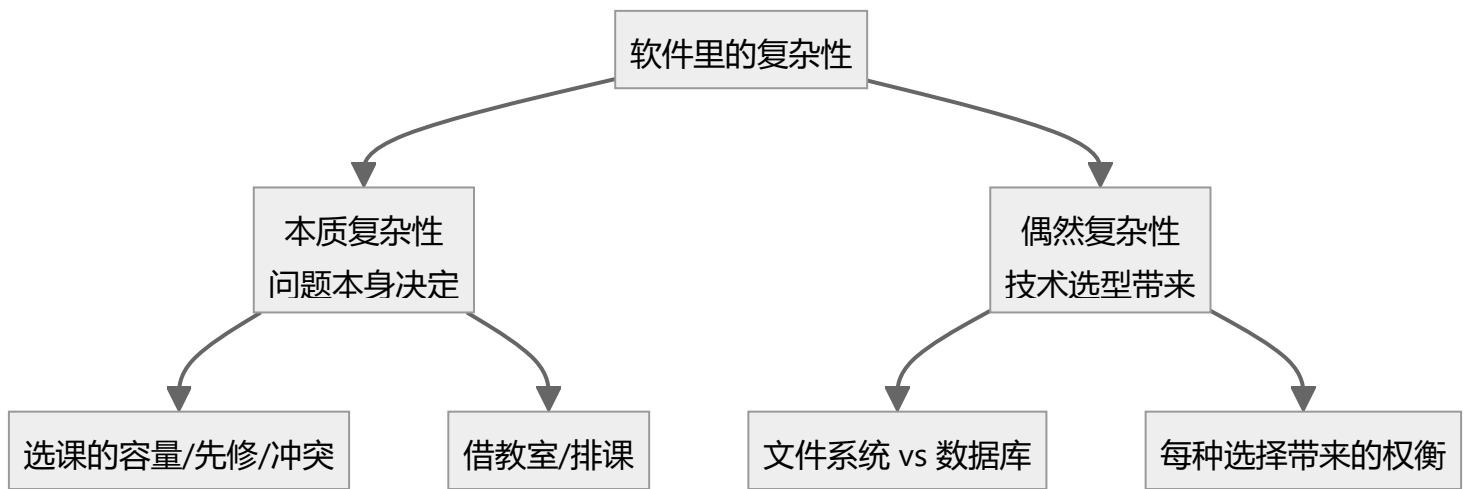
一、复习：需求与架构各自负责什么

1. 复杂性有两种，要分开看

Brooks 说过，软件工程的本质就是驾驭复杂性。这里有一个容易混在一起的点：**复杂性不是一种**。

一类是问题本身带来的。比如选课：要考虑课程容量、先修要求、时间冲突，还要排课、还要考虑教室，系统里老师和同学都能借教室。这个现实世界就是这么复杂，它在信息系统里的投影也一定这么复杂。这部分叫**本质复杂性**，没有办法消除，只能承认它存在。

另一类是技术选型带来的。比如同样存数据，用文件系统还是用数据库。文件系统管理起来方便，用 Unix 工具就能组合处理，但性能可能不如数据库，而且它不保证"读多个写多个"是原子的——数据库的事务能给你这个保证，代价是你每次要看数据都得先过一次数据库，多了一层中间层。这部分叫**偶然复杂性**，是可以设计、可以取舍的。



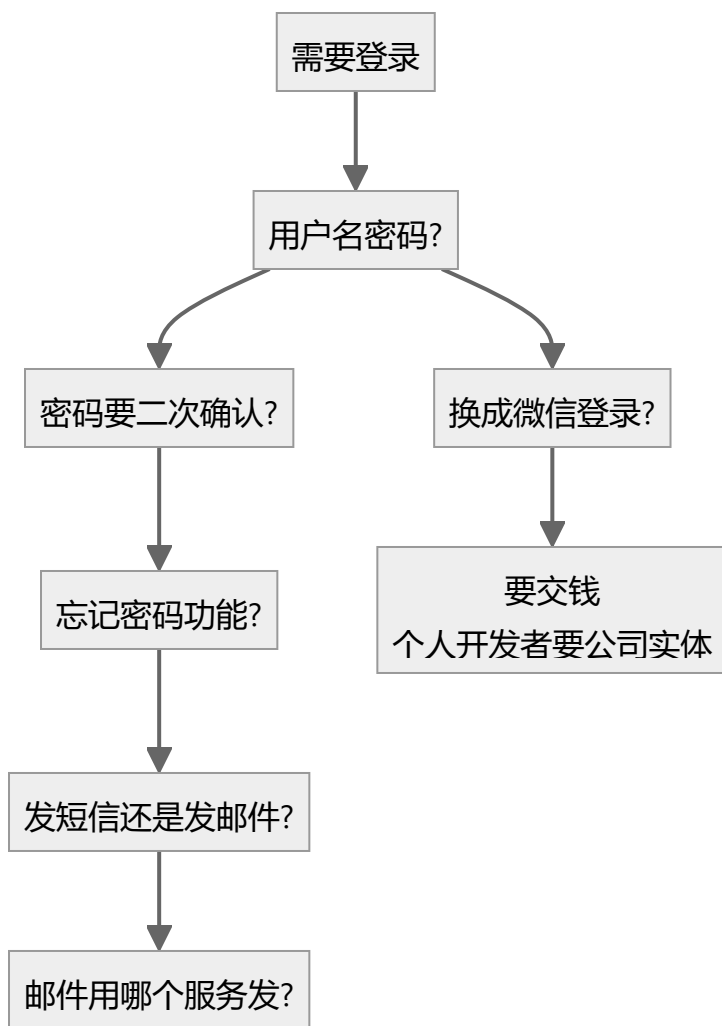
需求工程做的事情，是**直接面对本质复杂性**：承认系统就是复杂的，然后不停地和客户沟通，“我到底要什么”这件事要想清楚。架构做的事情，是**管理偶然复杂性**：把绕不开的复杂性隔离到不同模块里，用接口把模块切开。

2. 需求：深度优先和广度优先都要有

软件工程里讲的 MVP，可以理解成“深度优先”：先快速做出一小条能跑通的最重要功能，看看到底满足到什么程度，再决定需求要不要重做。

但只有深度优先不够。你的**思维方式也需要广度优先**：很多决策点要提前想过一遍。登录框用哪种方式实现、界面怎么布局、首页怎么展现，这些都属于需求的范畴。

登录这件事最能说明问题。你如果直接把需求交给模型说“我要登录”，它会猜一个：需要注册用户 → 用户名密码 → 第一次要确认密码 → 那要不要忘记密码？ → 不发邮件用户一定会忘密码 → 发短信还是发邮件？ → 邮件用什么服务发？ → 无穷的细节。



看到这条链，另一种选择就清楚了：干脆用微信登录，用户更方便。但代价是要交钱，而且对个人开发者不友好，往往需要一个公司实体才能注册。这就是一个典型的取舍。

所以最后要落到一个设计哲学上：**能不复杂就不复杂，能交给第三方管理就交给第三方**。当然也有反例——如果你就是想自己管用户数据、自己攒密码库，那是另一个方向（而且不合规）。深度优先和广度优先之间怎么平衡，没有现成办法，只能动手做、多看多想。

3. 需求和架构，都在“收缩可能性”

上一讲有一个图：从 Intention（意图）到 Specification（规格）到 Implementation（实现），每一步都会“飘移”。

- 一个意图，可以对应多种规格；
- 一种规格，可以对应多种实现。

需求和架构的作用，就是把这个可能性收缩掉：**需求收缩的是“意图空间”的不确定性，架构收缩的是“规格和实现空间”的可能性**。而你一旦做了确定，就背上了技术债，这个债要一直还下去。



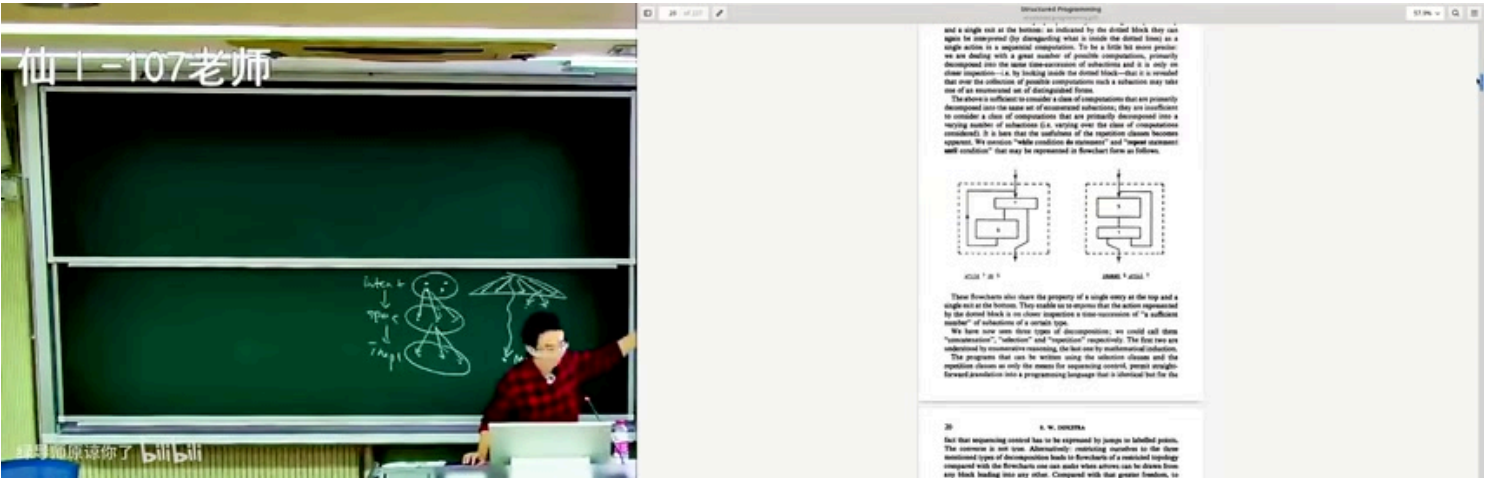
从十几行的小程序到上千万行的大系统，架构都无处不在。而且你在最开始什么信息都没有的时候就要做决定，一旦决定就影响之后所有走向。所以哪怕是做实验直接让 AI 生成一版，第一反应也应该是：**能不能回头再想想，有没有更好的方案。**

二、回到 1972 年：人脑怎么处理复杂性

这一讲花了不少篇幅读老论文，原因很简单：今天我们做的很多事情，过去的人已经想过一遍了。读原文能让你知道一个设计真正的出发点是什么，而不只是记住一个名字。

1. 一本老书：结构化程序设计

这是一本 1972 年的书，讲结构化程序设计。从今天的眼光看它不像一本现代软件工程的书——里面全是比较小的程序，还在讨论“只需要顺序、选择、循环这三种结构，就能表达任何程序逻辑”。

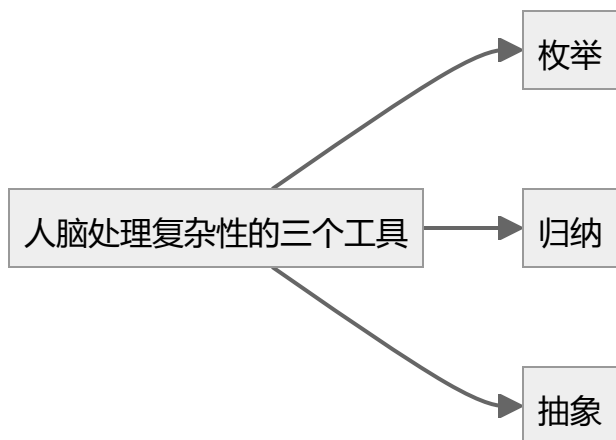


它的核心问题不是某个具体技术，而是一个更基本的问题：**人的理解能力是有限的，怎么让一个一次看不完的东西，变成可以逐层看完的东西。**

书里有一个我特别喜欢的比喻：你要做的程序，就像是做一条项链。项链上的每一颗小珍珠是一个小单元，把它们串起来，就得到一个完整的软件。这个比喻讲的就是分解——把一个人无法一次理解的东西，变成一个人可以逐层理解的东西。

2. 人脑处理复杂性的三个工具

书里有一个很重要的观点：**人脑能做的事情，归根到底就是三件。**



- **枚举**：把情况——列出来；
- **归纳**：从具体案例里找出规律；
- **抽象**：把相似的东西合并成一个更高层的概念。

后面讲的“抽象加层次分解”，本质上就是这三件事的组合。

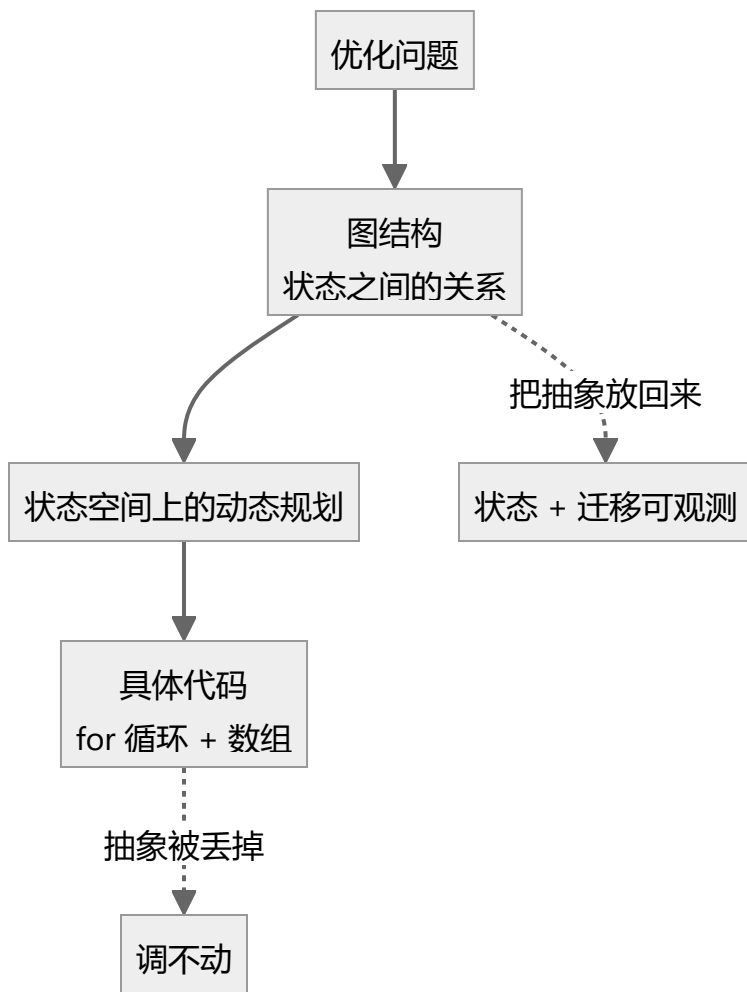
3. 一个现代例子：为什么动态规划难调

书里观察到“学生写的程序为什么这么烂”，原因是他们要在脑子里同时装太多东西。这个观察放到今天依然成立。

你写题的时候，什么题调起来最痛苦？往往不是模拟题，而是需要一点数学性的题，比如动态规划。模拟题你还能把它拆成几大块，每块封装成一个类，单独测试、单独调试，还能把程序中间状态映射到一个有物理含义的状态上。而一两百行的动态规划，你一旦错了，根本不知道从哪里调——因为中间全是纯粹的数学表示。

但这里其实有一层被扔掉的东西。我们谈 $dp[i][j][k]$ 的时候，实际上是在一个**优化问题**上工作，而这个问题本身是有图结构的，动态规划是在一个状态空间上做，满足无后效性，是一个有向无环图。这是关键的一层抽象。

而写代码的时候，这一层抽象被丢掉了。所以可以做的是把它**放回来**：把状态的概念还原出来，写一个 `get_descendants(i, j, k)`，让它返回这个状态所有可能的迁移。有了这层抽象之后，调试就变成了在看一个你能理解的结构。调完了，再改写成等价的、更高效的形式。这就是一个小规模上的架构。



4. 书里还提到了 class、object 和协程

这本书是 1972 年的，里面已经有 class、object 的概念。它还专门讨论了**协程**。

协程和线程很像。我上课时打过一个比方：线程就像人，共享内存就像物理空间。每个人有自己的记忆（我知道我做到哪一步，今天干什么，明天干什么），同时在共享空间里又有一些要合作完成的事。人和人之间的通信就成了一种协议。

这样你就可以把个工作分解成几条线，它们最终共同完成工作，中间通过数据结构或别的东西共享、通信。书里还举了下棋、编辑器这样的例子，并提出了“概念层次的金字塔”这个词。

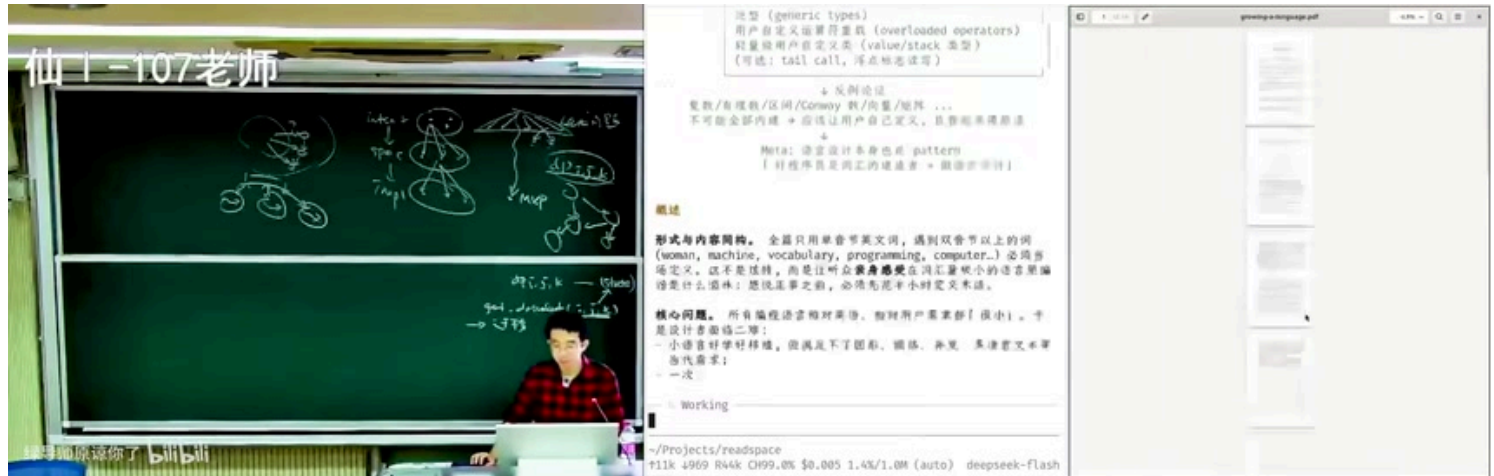
协程在今天更多是为了性能而实现的轻量级线程（比如 Go 的 goroutine 模型），切换和记录的开销很小。但如果你回到它被设计出来的那个时间点，会发现它真正想解决的问题，是**在软件世界里管理复杂性**。

三、1998 年：可生长的架构 —— Growing a Language

1. 一篇很特别的演讲

作者是 Guy Steele, Scheme 的设计者, 也是 Java 语言规范的参与者。他在 1998 年 OOPSLA 上做了一场名为 *Growing a Language* 的主题演讲。

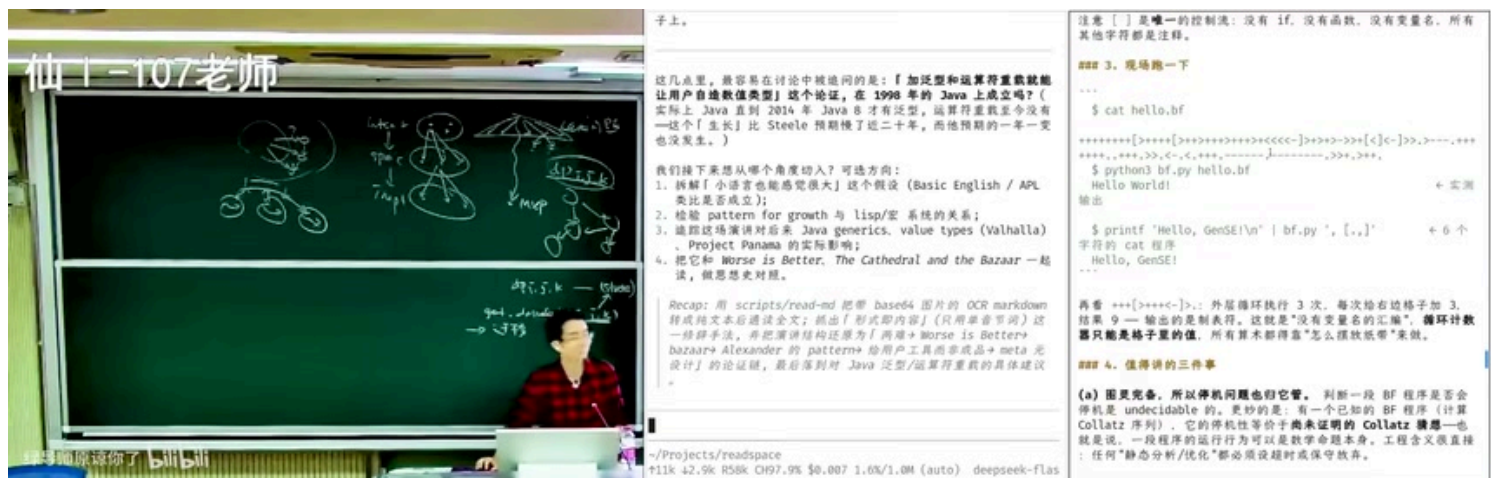
这场演讲很特别。它全篇只使用单音节的英文词，一旦遇到双音节以上的词，必须当场定义。这不是随便能想出来的形式——它本身就是一个表演性的设计。



2. 为什么编程语言必须小

演讲的引子指向一个现实问题：**所有编程语言，相对于用户需求来说都很小。**

原因是，编程语言要有形式语义，才能被机器执行。而一个语言如果变得非常大、包含很多复杂的文本概念，它的形式语义就定义不出来。所以语言必须小。但如果只是单纯地小，比如把一个图灵机直接编码成语法，那就小到人类根本读不了。



另一方面，语言小虽然好用，但满足不了不断膨胀的需求。而一次就把一个大语言设计好，体量太大、太慢，往往在成功之前就被某个小巧的语言取代了。

C 就是这个规则的典型例子。它非常成功，很大程度上因为它和机器几乎一一对应，没有多余的抽象，所以能用到几乎任何地方。但它也带来了设计上的问题：**它没有运算符重载**。你要表达“往队列里放东西”这种操作，写起来就很麻烦。



子。

这几点里，最容易在讨论中被追问的是：『加泛型和运算符重载就能让用户自定义数值类型』这个论证，在 1998 年的 Java 上成立吗？（实际上 Java 直到 2014 年 Java 8 才有泛型，运算符重载至今没有——这个「生长」比 Steele 预期慢了近二十年，而他预期的一年一变也没发生。）

我们接下来想从哪个角度切入？可选方向：

1. 拆解「小语言也能感觉很大」这个假设（Basic English / APL 类比是否成立）；
2. 检验 pattern for growth 与 Lisp/宏系统的关系；
3. 追踪这场演讲对后来 Java generics, value types (Valhalla)、Project Panama 的实际影响；
4. 把它和 *Worse is Better*, *The Cathedral and the Bazaar* 一起读，做思想史对照。

Recap: 用 scripts/read-md 把带 base64 图片的 OCR markdown 转成纯文本后通读全文；提出「形式即内容」（只用单语单词）这一特殊手法，并把演讲结构还原为「两端→Worse is Better→Bazaar→Alexander 的 pattern→给用户工具而非成品→meta 元设计」的论证链，最后落到对 Java 泛型/运算符重载的具体建议。

```
NAME
    qsort, qsort_r - sort an array

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <stdlib.h>

    void qsort(void base[.size * .rmemb], size_t rmemb, size_t size,
               int (*compar)(const void [.size], const void [.size]));
    void qsort_r(void base[.size * .rmemb], size_t rmemb, size_t size,
               int (*compar)(const void [.size], const void [.size], void *),
               void *arg);

Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

qsort_r():
    _GNU_SOURCE

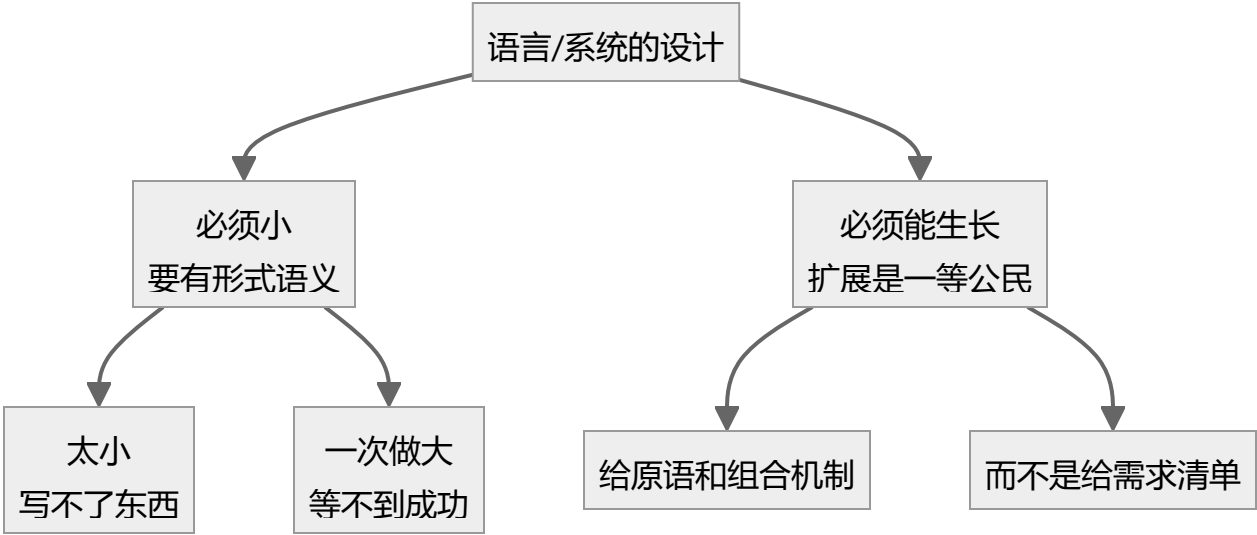
DESCRIPTION
    The qsort() function sorts an array with rmemb elements of size size. The base argument points to the start of the array.

Manual page qsort(3) line 3 (press h for help or q to quit)
```

qsort 就是一个很好的例子：它要一个比较函数的指针，还要元素个数和元素大小两个参数，名字又长又不好记。C 没有能力让你自定义比较好的语言抽象，所以表达任何东西都容易遇到困难。

3. 语言该怎么长大

演讲提出的核心主张是：**不要一次转移，而要生长。**



这和软件需求是同一回事：一个好的架构能抵抗需求的变更。当世界变化的时候，你能提前看出一点未来会发生什么；如果只针对现在设计，就可能陷入技术债的泥潭。**能生长的架构才是最好的架构。**

它举了几个语言的例子。从 C 到 C++ 是一次"生长", 但我觉得长得并不好——为了实现各种功能, 模板被滥用, 很多代码没人看得懂。模板本身是好技术, 但长出了比较奇怪的东西。

演讲还给 Java 提了三条建议: 加泛型、加运算符重载、加轻量级用户定义类型。没有泛型, 构造抽象时就会像 C 或早期 Java 那样到处是强制类型转换; 运算符重载 Java 到今天也没加; 轻量级用户定义类型现在有了。

4. 一个更深的判断标准

这一节最值得记的是这几句话:

- **扩展必须是一等公民。** 不是说"我有一张需求清单, 把它全部实现掉就叫正确", 而是你的系统要能够被扩展。
- **扩展之间要能组合。**
- **给原语和组合机制, 而不是给需求清单。** 照着需求清单写实现也可以, 但如果能把需求分解成几个跳出具体问题的概念, 概念又能变成具体的领域对象, 还能组合起来, 那就可能是一个对的设计。

这里还有一个对做 OJ 的提醒。从进大学开始疯狂做题, 训练出的是"解题机器"的习惯: 题对了就行, 代码不需要维护, 也不需要扩展。这会让人不自觉地偏向"用户一次性代码"的思路, 思考扩展的空间就变少了。这门课在某种程度上是要把这一点拉回来。

我甚至期待一个"大学毁灭计划": 从大一第一门程序设计课开始, 把知识点全拆散, 不用某本教科书, 把 1970、1980、1990 年代的经典材料全部拆开, 重新为你生成一份教材。

四、三个经典架构案例

上面讲的是原则, 下面看三个具体的、经得起时间检验的架构。

1. Unix: 文本接口 + 管道

Unix 哲学有三条:

1. 每个工具只做一件事, 并把它做好;
2. 程序之间可以协同工作;
3. 程序处理文本流。



设计解空间：UNIX Philosophy

三条原则

- Do one thing and do it well.
- Write programs to work together.
- Write programs to handle text streams, because that is a universal interface.

精髓是第三条

- 文本接口统一了人-程序、程序-程序的接口，使用者可以
 - 自由组合程序
 - 随时检查程序的中间结果
-  McIlroy 在 1964 年就提出：像连接花园水管一样连接程序。原始备忘录

第三条是最精髓的设计。因为**文本接口同时统一了人和程序、程序和程序的接口**。如果只是二进制字节流，对人就很不友好。

这一点和用数据库的感觉正好相反。数据库很好，但你总觉得隔了一层纱：每次查东西都要写一句 SQL，而这句 SQL 对你来说是间接的。人其实不太适应做任何事都要加一层间接——模型可以，比如让它每说一句话都先“想一下”，但让人来做这种间接指令，就会很痛苦。

这个想法在 1964 年就已经出现了。当时总结了四件事：**像接花园水管一样连接程序**、动态链接和库的索引、用统一的命令行参数、用退出状态告诉下一个程序发生了什么事。管道是 1983 年才引入 Unix 的。

Unix 靠的是一套世界里的“语言”：所有命令行工具都遵循大致一样的参数约定，输出格式也尽量保持一致，还有人取过名字叫“Unix 输给各种吐槽”——意思是这些设计其实有很多可批评的地方，但正是这套统一标准，让人好理解，也让工具之间能互相配合。有了 sed、awk 这类工具打底之后，程序的输出逐渐向“对这几个工具友好”的方向发展，也就是同时对人也对工具友好的方向。

这里我最喜欢的一个例子是“打开文件菜单”。

你写一个软件，需要一个打开文件的菜单，要么自己实现一个，要么用操作系统的。如果你对这两个都不满意，你还有别的办法吗？在传统 GUI 里，几乎没有。

我在 Windows 或 Emacs 里用任何一个编辑器，都可以给它配一个自己的“打开文件”菜单。比如用 Nano，配好之后按下去，它就弹出一个打开文件的菜单——而且这个菜单是我自己定义的。

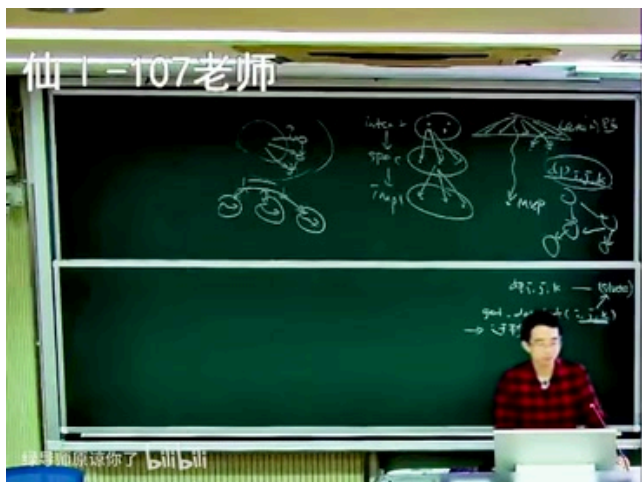


这就是**工具的组合性**。把共享的文本接口、编程语言、Unix 管道放在一起，你就有一个可以无限组合、无限复用的空间。所以我上操作系统课时说：你们在 Unix 里，其实每个人都是程序员，因为你敲的每一个命令都是一个小程序，而它负责把各种各样的工具组合起来。这种可组合性，到今天都很难在一个 GUI 程序里得到。

2. 关系数据库：把逻辑关系抽出来

Codd 在 1970 年的论文里提出了关系模型。它的起点是：**用文件系统管理数据，你面对的是一个很原始的接口。**好处是 Unix 世界的工具可以复用，坏处是太自由，没有统一的管理。

关系模型做的事情，是**对世界建立一个模型**：世界上的数据可以用二维表来表示，表和表之间再用连接之类的操作来处理。数据怎么存、可靠不可靠，全部交给底层的数据系统。在这个模型之上，长出了 SQL 这门结构化的查询语言。SQL 和 shell 一样，本质上也是“一门语言”，是一个接口，足够灵活，就会长成一门编程语言。



仙一-107老师

案例二：关系数据库

“ A relation model of data as a basis for protecting users of formatted data systems from the potentially disruptive changes in data representation caused by growth in the data bank and changes in traffic. (E. F. Codd, 1970)

把物理世界状态投射到关系数据库中

- 分离“数据的理解”与“数据在机器中的组织”
- “谁借了哪本书”保持不变。数据页、索引、访问路径可以改变。Codd 的数据独立性

在逻辑关系上构建应用

- SQL、查询优化器、事务实现使 CRUD 成为标准的软件架构 pattern
- “随后发展的关系数据库系统，把“声明要什么”落实为可选择的执行计划。System R (1976)

有了 SQL、查询优化器、事务之后，事情就变得很干净：**二维表在任何时刻都是当前世界的一个状态**，一次事务把状态带到下一个状态，而视图是“用查询对当前状态的一种理解”。

更重要的是，数据库的实现给了我们近乎无限扩展的能力。到今天，SQL 数据依然可以无限地扩展。所以数据库带起了整个人类世界数据信息化的一波浪潮：学籍可以信息化，财务可以信息化，电商订单可以信息化。因为它

表示的是**实体和实体之间的逻辑关系**——只要逻辑关系理清楚，现实世界就能在信息系统里落地。它只规定"世界里有表、要有一个基础的 schema"，但 schema 具体是什么，由你决定、由应用定制。

这是一个非常顶级的、隔离复杂性的抽象，而且有无限的扩展能力。

3. Internet：只做好一件事，其余交给别人

Internet 也是一个典型的可生长架构。它解决的问题非常简单：**连通性**。

RFC 1958（1996 年 6 月的《互联网架构原则》）里有一句核心的话：唯一应该永存的原则，就是变化本身这个原则。



它只做一件事：把网络连起来，让上面的应用能爆发出来。比如用 TCP 的时候，你再也不用担心数据传送可靠不可靠，无论什么数据流都能稳定可靠地从一端送到另一端。至于上面跑什么应用，能支持就支持，不做预设。

当然，连接这一层本身也有不少细节：流量控制、连接的管理、什么时候判断连接中断，这些都是架构里必须包含的设计。其他的就基本全部交给别人。

在这个基础上，上面的世界其实变了很多。我读书的时候还没有全网 HTTPS，大家都是在 HTTPS 上"裸奔"的，直接用 Telnet 连到学校 BBS 去灌水；传文件用 FTP，因为 FTP 协议本身就支持断点续传，在校内下电影下得很开心——网络不稳定，下次可以接着传。今天 Telnet 和 FTP 因为安全性都不够用了，我们把很多功能直接实现在 HTTPS 一个接口里，HTTPS 就成了 TCP/IP 之上的又一层。今天你访问几乎所有微服务，全都是 HTTPS，再配上一套 RESTful API 的规范（虽然是建议性的，可以不严格遵循）。

在共同的规则之上，局部可以持续更换——这就像先建好了一个城市的交通网络，然后城市里的建筑一直在向前演进。

五、应用层的架构：从数据库中心到 MVC

系统这一层的地基打好了（操作系统、数据库、Internet），上面就能盖各种业务系统了：教务系统、图书管理系统，等等。

如果让时间回到 1998 年或 2000 年，让你为学校或企业设计一个软件系统，你手上有的就是 Internet、数据库、操作系统。最直接的想法是：**为每一个页面或每一个功能，都做成一个数据库事务。**

这其实和需求是对应的。数据库已经完成了物理世界在信息世界里的"投影"——它就是一堆表的分解。数据库的事务驱动状态变化，而软件工程做的事情，就是把需求分解成不同的需求点，每个需求点变成一个数据库操作；对当前状态，再用各种数据库视图去切一刀拿出来。这就是最经典的应用程序架构，在对每个需求点都是数据库查询的场景里，它非常好用。

让这件事变得前所未有的容易的，是模板语言。很早的时候，PHP、JSP 就开始探索"怎样在页面里嵌入服务端脚本"了。比如早期在 HTML 里写脚本，甚至可以循环、可以判断。有了这个能力，一个页面只需要一个数据库事务把所有数据捞出来，放进一些 List、Map 里，再用模板的方式投射成 HTML 页面。再加上 HTML 里 DOM 和样式分开（CSS），可以摆出几乎任何形式的界面。所以当时各种管理系统就集中式地爆发了。

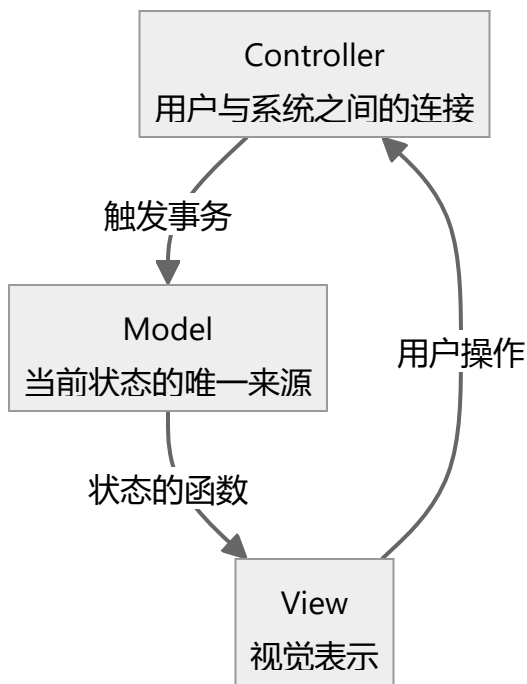
用今天的例子说就更直观了：你要写一个 AI 应用，需要定制 prompt，怎么拼？最笨的办法是 `prompt = A + B + C + D` 这样一路加下去。更好的办法是用模板语言，把 prompt 写成一个带变量的模板，比如 `instructions` 变量，渲染的时候直接贴进去，还能传入今天的日期。这样 prompt 就好维护了。

MVC：一个非常干净的架构

这套"每个页面是一个数据库事务"的想法，导出了一个非常重要的模式：**MVC (Model-View-Controller)**，最早是 1979 年 Smalltalk 时期的，只有两页纸。

- **Model** 表示知识，是当前世界知识的单一来源——谁喜欢什么、有哪些订单。
- **View** 是 model 的视觉表示。它是状态的函数，而且应该是一个纯函数，不应该有改变状态的能力。
- **Controller** 是用户和系统之间的连接：它把用户的输入变成对相关 view 的呈现，也把用户的输出（比如菜单或数据的含义）呈现给用户。

说白了，MVC 把"能修改的点"变成一层事务，把"你看到的东西"变成状态的一个投射。



放到教育系统里就很自然：Model 是课程、选课记录；View 是你看到的那张页面；Controller 是各种按钮、下拉框、输入框——你点一下，它对应地进入一个数据库事务。

视图状态：为什么会有 MVVM 和 React

MVC 有一个前提：整个页面是单一 HTML 页面，每次点按钮，页面会重新刷新、重新更新。在这个前提下它是成立的。

但今天的界面上有大量元素，它们的状态**不在 model 里**：

- 可以随时展开的菜单；
- 一个搜索框，输入的过滤条件；
- 鼠标移过一个元素时它应该亮起来。

这些都需要状态，但这些状态不应该存在数据库里。于是就有人想：为什么不把"视图状态"也当成一个 model？这就发明了 **MVVM**。

React 更纯粹一点：它希望你保留 model（网页上看到的東西是数据库的一个投射），然后在 view 上面再加一层状态——鼠标是不是 hover 了、输入框的内容状态是什么。输入框和这个状态绑定，状态随时可以改。更进一步，它问：能不能直接写一个函数，把界面上所有状态作为输入，把 DOM 树作为输出算出来？

理论上可以，但有一个严重问题：**性能**。一个网页可能有几万个元素，当你往里输入一个字符，整个状态全部要重算。所以 React 在系统内部做了大量繁重的工作，把这些负担藏起来——你只按照它的模型声明"状态是什么、状态怎么变"，剩下它帮你更新，内部维护一张依赖关系图。

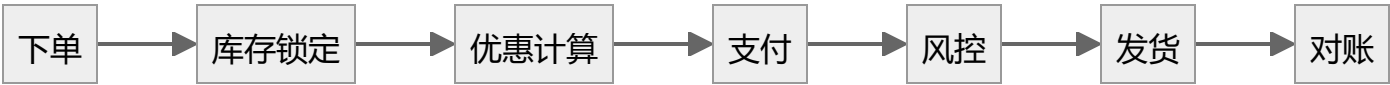
Vue 和 React 的思路差不多，随着版本迭代两者也越来越像。它们共同的设计理念是：**隔离复杂性，把变的东西和不变的东西分离，把逻辑和数据源头分离。** 因为我们"头很小"，关注状态的时候就别同时关注它是怎么变成画面的；等状态完全确定了，再看怎么把它变成视图。

这两个框架都是由状态决定显示，只是"更新的模型"稍有不同。但它们并没有解决真正的本质复杂性——尤其是面对长流程业务的时候。

六、长流程业务的难题：状态字段不够用

1. 淘宝订单说明了什么

假设你要实现一个淘宝。下单本身没问题，但下单之后会牵出一整条流程：库存要锁定、要计算优惠、要支付（而且支付需要用户确认）、支付完要风控、要发货、要退款、要售后、要对账。



前面我们说"用当前状态表示世界"，这在这里其实做了一个不切实际的假设。当前状态当然可以用二维表表示，问题在于：**我怎么表示一个订单进行到哪一步了？**

最直接的办法是加一张 Order 表，里面有一列叫 status，一代表下单、二代表库存、三代表什么什么；或者把状态展开成多个字段，是不是已下单、库存有没有锁定、支付了没有。一旦这样，状态机就非常复杂了，而且你会遇到更麻烦的问题：在某个节点上，比如触发了风控之后，你需要设计"返回"的逻辑。

这就是本质复杂性。业务系统越长越大，问题就越明显。

2. 一个真实世界里的现象：数据不一致

你会发现，**有的学生会处于既可以毕业、又不能毕业的状态。** 因为他挂科的时候，程序可能有 bug，或者有一次特批导致某个地方被 override 了——本来风控应该触发，但没有触发；或者本来该被拦下的，被另一个人 override 了。系统里因为逻辑错综复杂，最后数据就变成"风控失败"却进入了下一个环节。

这种数据到底是什么？你怎么解读？

甚至很多数据是**静默损坏**的——没有报错，只是不对。我相信学校的教育系统里就有大量这样的数据：因为经常手工操作，教务系统里有很多不一致的数据在，但好像最后也没有影响这个同学毕业，就只能不管了。

根源在于：**业务流程可能是一整条很长的流程，它既要编码过去，又要编码现在，又要编码未来。** 本质上，当前状态是"现在的一个快照"，而这个快照是由过去推演过来的——但"它怎么来的"这个过程丢了。过程一丢，不一致就很难查。

3. 从记账得到的启发：复式记账

先看一个更简单的例子：你有没有记过账？

如果你要真正严肃地记账，会计上有一个要求叫**复式记账法**。它就是为了防错——当数据出现不一致的时候，它能够被检查出来。它要求每一笔交易都同时记入两个或两个以上的账户，且金额相等、方向相反。

相比之下，只记一条虽然简单，但无法反映业务的完整结构，也不具备自平衡的校验能力。复式记账还支持“试算平衡”。

人类发明这套东西，正是因为**一个当前状态很可能是损坏的、不对的，需要有一种方法把“为什么不对”找出来。**

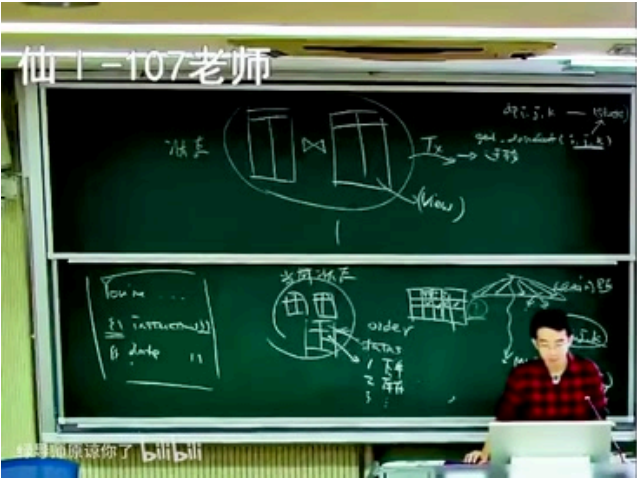
4. 用流水账代替当前状态

假设我现在有一个记账需求，要生成一张资产负债表——大概长这样，有现金、存货、固定资产、应付账款、短期借款、所有者权益等等。如果要以符合会计法规的方式记账，就必须得到这样一张表。但我对会计没有任何知识。

这时候我换了一个思路：**不再记录当前状态，而是记录所有状态的迁移。**也就是说，我不记录“现在是多少”，我记录“一笔流水账”——今天进了一千块，明天又出了一千块，为什么进的、为什么出的。这样我其实就是在记流水账。

我让 AI 直接实现了一个小系统，而不是先去设计一个数据库。它的结构是这样的：我有很多记账动作，这些动作是从最终要交付的资产负债表里所有可能科目中提取出来的。比如我要付一笔营业成本的钱，每付出一份，货币资金会怎么变、所有者权益会怎么变、应付账款会怎么变、现金流量表上会怎么变——这些都由 AI 生成的表格来描述。

仙一-107老师



Recap: 通过并连接翻译了 Reenskaug 1979 年 MVC 原始备忘录。核心记忆点是三条职责规律——模型层表达知识，视图只负责展示该知识且不懂输入，控制器不对视图做补充；以及 editor 作为可插拔中间层的思想。

open it with evince

```
$ cd /home/jyy/Projects/GenSE/p7/lect/ 56 (evince 1979-12-MVC.pdf >/dev/null 2>&1 &); sleep 1; pgrep -a evince (timeout 30s)
```

14196 evince 1979-12-MVC.pdf
14204 /usr/libexec/evince

Took 1.0s

已用 evince 打开 1979-12-MVC.pdf (PID 14196)。

Recap: 用后台方式启动 evince 打开 PDF，并用 pgrep 确认进程存活，避免阻塞终端。

~/Projects/GenSE (main)
11.4k 41.3k R20k CH97.5% \$0.002 0.5%/1.0M (auto) deepseek-flan

资产	金额	负债与权益	金额
现金	50	应付账款	30
存货	80	短期借款	70
固定资产	170	所有者权益	200
合计	300	合计	300

两边永远相等——这就是复式记账保证的平衡。

一个直觉理解

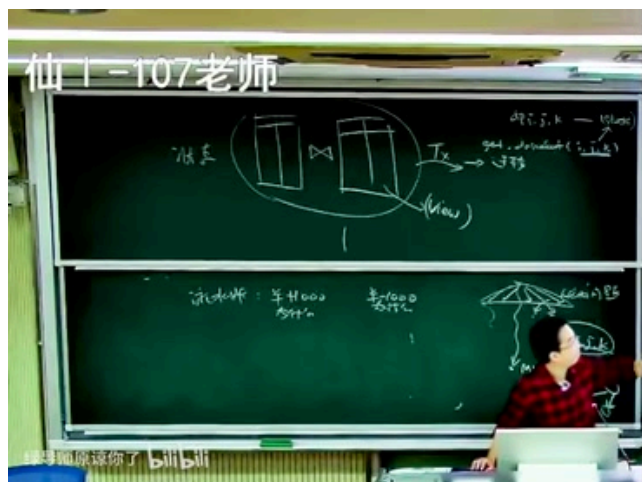
把「资产负债表」想成给企业拍的一张X光片：

- 左边（资产）：钱变成了什么形态
- 右边（负债+权益）：钱是从哪来的——借来的（负债）还是股东投的/赚的（权益）

同一笔钱，左边看是「用途」，右边看是「来源」，总金额当然相等。

和其他报表的关系

- 资产负债表：时点快照（12月31日这一刻，公司是什么样）
- 利润表：一段时期的经营成果（这一年赚了多少钱）
- 现金流量表：一段时期现金的进出



资产负债表	利润表	现金流量表
资产合计	+1.00	营业利润
负债合计	-1.00	利润总额
所有者权益合计	-1.00	净利润
资产合计	+1.00	管理费用
负债合计	+1.00	业务招待费
所有者权益合计	+1.00	营业利润
	-1.00	利润总额
	-1.00	净利润
资产合计	+1.00	管理费用
负债合计	+1.00	研究费用
所有者权益合计	+1.00	营业利润
	-1.00	

然后我做了一个关键的架构设计：**用一个小语言把每条流水账条目编译成一个操作**，并且这个历史是不可篡改的（persist history）。哪怕我要特批让某个不能毕业的同学毕业，我也必须在这里写一条 persist，说明“我要允许某某某毕业”。

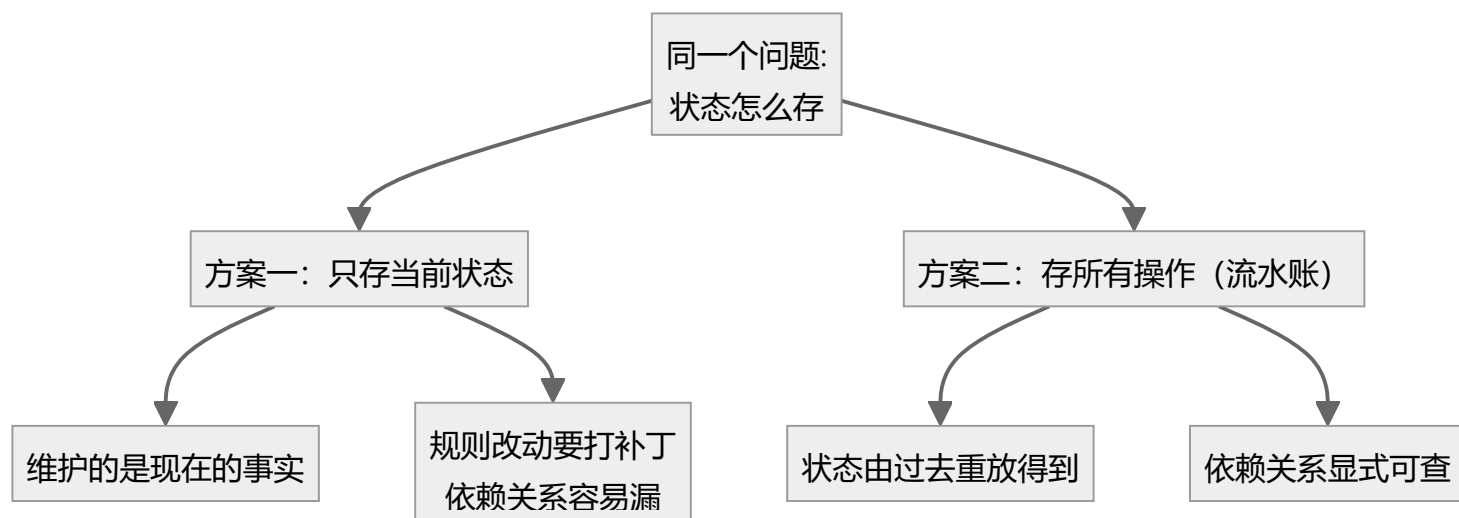
当前的状态，就是**由所有操作 reduce（重放）得到的**。每个操作都会说“这个资产负债表要怎么改、第二张表要怎么改”，把所有操作按时间顺序 replay 一遍，就得到当前可以提交出去的资产负债表。

这里还有一个小的实现技巧：一开始我写了一段非常长的代码，很糟糕；后来用了一个常见的函数式编程技巧——给解析器加上一个参数（相当于柯里化），让参数对应的状态先确定下来，剩下的是一个函数，代码就组合起来了。

好处是：记账从此不再是问题。比如银行收支隔一段时间要整理一下，把报表上体现的所有东西按时间顺序放进去，就能得到当前的报表。刷新一下即可。

5. 两种方案到底差在哪

把两种方案放在一起看：



方案一（数据库存当前状态）的困难：

数据库管理的是"现在"——现在学校里的状态是什么，每个学生是什么，学校有哪些课程，每个学期有哪些课程。它代表一个当前事实，而这个状态是**可变的**。

状态可变，就意味着一旦你的逻辑有错、或者规则改变，状态就很容易变得不一致：你会从一个对的状态走到一个错的状态。

而好的架构应该是能生长的，能容纳"突然多了一个新需求""过去的某条规则改掉了"这种情况。当过去的规则改掉时，你就需要为状态生成一个补丁——而数据库的迁移（为状态打补丁）是一个非常危险的操作。

举个例子：成绩更正。成绩从及格改成不及格，会带来大量后果。你的学分数要减一，而学分数又会影响你能不能毕业、能不能准出、能不能选下一门课。假设你已经选上了某门课的先修课，现在系统里记你挂科了，你还能选先修课要求的下一门课吗？数据库内部其实已经不一致了。本质复杂性里有大量这种相互依赖：我依赖你，你依赖我。

在这种依赖错综复杂的情况下，如果只存当前状态，**你到底影响了哪些地方，是很难评估的，很容易漏掉。**

方案二（流水账/事件溯源）的好处：

你做的事情不是维护一个当前状态，而是搞清楚所有原始数据之间的依赖关系——从一个记账条目到一个操作。这个翻译过程也可能出错，但出错以后，比如当期账会有人工来试算平衡、审核，审核发现错了就直接改过来，不付出任何代价。

如果发生的是真正的影响，比如一门课的成绩翻转影响其他课程，因为你不维护当前状态，你可以把所有计算重新做一遍。哪怕只是做一次自动的数据流分析，你都知道"通过了某门课"这个事实从通过变成不通过，可能会影响哪些结论。

因为数据流被显式建模出来了，而不是以状态的形式藏在当前状态里。

而且因为它是 replay-based 的，当前状态完全由过去决定，所以你还可以用它来**推演未来**。比如一个流程现在处于"支付中"，你可以直接推演出"如果风控成功，状态会到哪；如果风控失败，状态会到哪"。如果我有一个状态，我就能在这个基础上评估"我到底会不会破产、破产概率有多大"——只要有一个模型，只要知道应收账款有多少回不来就会破产，就能对未来做评估。这些都是一个好的架构能带来的好处。

所以流水账式的记录在解决这些问题时会显得更好一些。它没有办法消除本质复杂性——课程的依赖关系、毕业的依赖关系，这些事实本来就存在——但它把**事实、视图和预期分开了**。如果世界没有按你的预期发展，你就通过追加新的事实来处理，而不需要让工程师去手改一个数据库，让它进入越来越糟糕的状态。

你可以追加一条："我现在认为这个同学可以毕业，其他所有东西都不管了。"因为有条记录，他就可以毕业。这就相当于：你有一份可查证、可溯源的机上过程，等于显式地建模了系统里的各种数据依赖。

所有的计算逻辑都是可以溯源的：我通过了一门课，所以学分要加上这门课的学分，就要取出当前这门课的学分。有了流水账，你就知道在当前这个时刻，计算机系统基础课的学分是几个，不会出现“确实减了学分，但学分还够不够毕业”这种说不清的问题。

6. 关系数据库的一个根本局限

最后要指出关系模型的一个问题。

Tony Hoare 调侃过自己发明空指针是一个“十亿美元的错误”，因为大量的程序崩溃都和它有关。关系数据库也有一个类似的问题——我不想说是设计缺陷，因为在当时它确实是一个非常好的、可扩展的架构，但它在管理数据的时候，**没有把任何图灵可计算的计算关系作为一等公民。**

什么意思？比如学生表里有一个字段叫“能不能毕业”，值是 0 或 1。几乎所有的数据库都会这样设计，因为你必须能快速把它查询出来——关系代数没法建模一个非常复杂的计算。

但这个 0 和 1，就成了数据库里一个潜在的**不一致点**。为什么？因为它是从别的数据推导出来的（derived state），可它被当成了一个独立的事实存了起来。当底层数据变化时，这个字段不一定同步。

现在的框架都在用各种方式找补：memo、React、数据流，各种把 derived state 找回来的手段，本质上都是在解决同一个问题。

一句话总结

- **复杂性要分开看**：问题本身的复杂性（本质）绕不开，只能承认并用架构隔离；技术选型带来的复杂性（偶然）是可以设计和取舍的。
- **需求收缩意图，架构收缩实现**：确定得越多，技术债背得越多。
- **人脑容量有限**，处理复杂系统的办法是抽象加层次分解——枚举、归纳、抽象三件事。
- **好的架构能生长**：扩展是一等公民，给它原语和组合机制，而不是一张需求清单。
- **经典案例的共同点**：只做一件事，把接口统一（Unix 的文本流、数据库的逻辑关系、Internet 的连通性），剩下的交给别人去长。
- **应用层从数据库中心演化到 MVC**，再因为视图状态的出现演化出 MVVM 和 React，核心始终是把变和不变分开。
- **长流程业务要小心当前状态**：把过程丢了，不一致就无法追溯；记录所有变更再重放，能让依赖关系显式起来，还能用来推演未来。